

Padrões para a Camada de Dados

```
graph TD; P[Presentation] --- BL[Business Logic]; BL --- DS[Data Source]; DS --- DB[(Database layer)];
```

Presentation

Business Logic

Data Source

Database layer

Catalog of Patterns of Enterprise Application Architecture

Last Significant Update: January 2003

A short summary of the patterns in Patterns of Enterprise Application Architecture (P of EAA).

These pages are a brief overview of each of the patterns in P of EAA. They aren't intended to stand alone, but merely as a quick aide-memoire for those familiar with them, and a handy link if you want to refer to one online. In the future I may add some post-publication comments into the material here, but we'll see how that works out.

Many of these diagrams demonstrate the rather poor GIF output of Visio. The nice diagrams were redrawn for me by David Heinemeier Hansson

Domain Logic Patterns: Transaction Script (110), Domain Model (116), Table Module (125), Service Layer (133).

Data Source Architectural Patterns: Table Data Gateway (144), Row Data Gateway (152), Active Record (160), Data Mapper (165).

Object-Relational Behavioral Patterns: Unit of Work (184), Identity Map (195), Lazy Load (200)

Object-Relational Structural Patterns: Identity Field (216), Foreign Key Mapping (236), Association Table Mapping (248), Dependent Mapping (262), Embedded Value (268), Serialized LOB (272), Single Table Inheritance (278), Class Table Inheritance (285), Concrete Table Inheritance (293), Inheritance Mappers (302).

Object-Relational Metadata Mapping Patterns: Metadata Mapping (306), Query Object (316), Repository (322).

Web Presentation Patterns: Model View Controller (330), Page Controller (333), Front Controller (344), Template View (350), Transform View (361), Two-Step View (365), Application Controller (379).

<https://martinfowler.com/eaCatalog/>

Padrões para as várias camadas

Camada de Apresentação	MVC Front Controller Page Controller Page template
Camada de Negócio	Service Layer
	Domain Model
	Table Module
	Transaction Script
Camada de Dados	Row data gateway
	Table data gateway
	Active Record
	Data Mapper

Presentation

Business Logic

Data Source



- **Relational Databases** (Postgres, MySQL, Oracle, ...)
- Non-relational Databases (Cassandra, MongoDB, ...)
- Messaging Systems (Redis, Kafka, ...)
- External Systems (SOAP, REST APIs, GraphQL, ...)

JDBC

Java **D**ata**B**ase **C**onnectivity

Data Source

JDBC API



DriverManager	Factory para criar ligações à base de dados
Connection	Representa uma ligação de um cliente à base de dados, para executar transações
Statement	Comando SQL que é executado dentro de uma Connection
PreparedStatement	Comando SQL que é parametrizado pelo utilizador (+eficiente e +seguro)
ResultSet	Estrutura que guarda o resultado de um query (cursor!)

JDBC

Java **D**ata**B**ase **C**onnectivity

Data Source

JDBC API



DriverManager	Factory para criar ligações à base de dados
Connection	Representa uma ligação de um cliente à base de dados, para executar transações
Statement	Comando SQL que é executado dentro de uma Connection
PreparedStatement	Comando SQL que é parametrizado pelo utilizador (+eficiente e +seguro)
ResultSet	Estrutura que guarda o resultado de um query (cursor!)

JDBC

Java **D**ata**B**ase **C**onnectivity

Data Source

JDBC API



DriverManager	Factory para criar ligações à base de dados
Connection	Representa uma ligação de um cliente à base de dados, para executar transações
Statement	Comando SQL que é executado dentro de uma Connection
PreparedStatement	Comando SQL que é parametrizado pelo utilizador (+eficiente e +seguro)
ResultSet	Estrutura que guarda o resultado de um query (cursor!)

JDBC

Java **D**ata**B**ase **C**onnectivity

Data Source

JDBC API



DriverManager	Factory para criar ligações à base de dados
Connection	Representa uma ligação de um cliente à base de dados, para executar transações
Statement	Comando SQL que é executado dentro de uma Connection
PreparedStatement	Comando SQL que é parametrizado pelo utilizador (+eficiente e +seguro)
ResultSet	Estrutura que guarda o resultado de um query (cursor!)

JDBC

Java **D**ata**B**ase **C**onnectivity

Data Source

JDBC API



DriverManager	Factory para criar ligações à base de dados
Connection	Representa uma ligação de um cliente à base de dados, para executar transações
Statement	Comando SQL que é executado dentro de uma Connection
PreparedStatement	Comando SQL que é parametrizado pelo utilizador (+eficiente e +seguro)
ResultSet	Estrutura que guarda o resultado de um query (cursor!)

JDBC

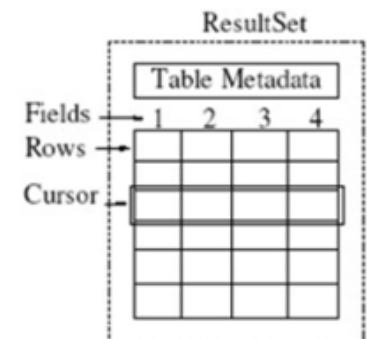
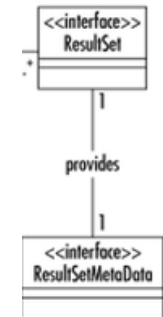
Java DataBase Connectivity

Data Source

JDBC API



DriverManager	Factory para criar ligações à base de dados
Connection	Representa uma ligação de um cliente à base de dados, para executar transações
Statement	Comando SQL que é executado dentro de uma Connection
PreparedStatement	Comando SQL que é parametrizado pelo utilizador (+eficiente e +seguro)
ResultSet	Estrutura que guarda o resultado de um query (cursor!)



JDBC

Java DataBase Connectivity

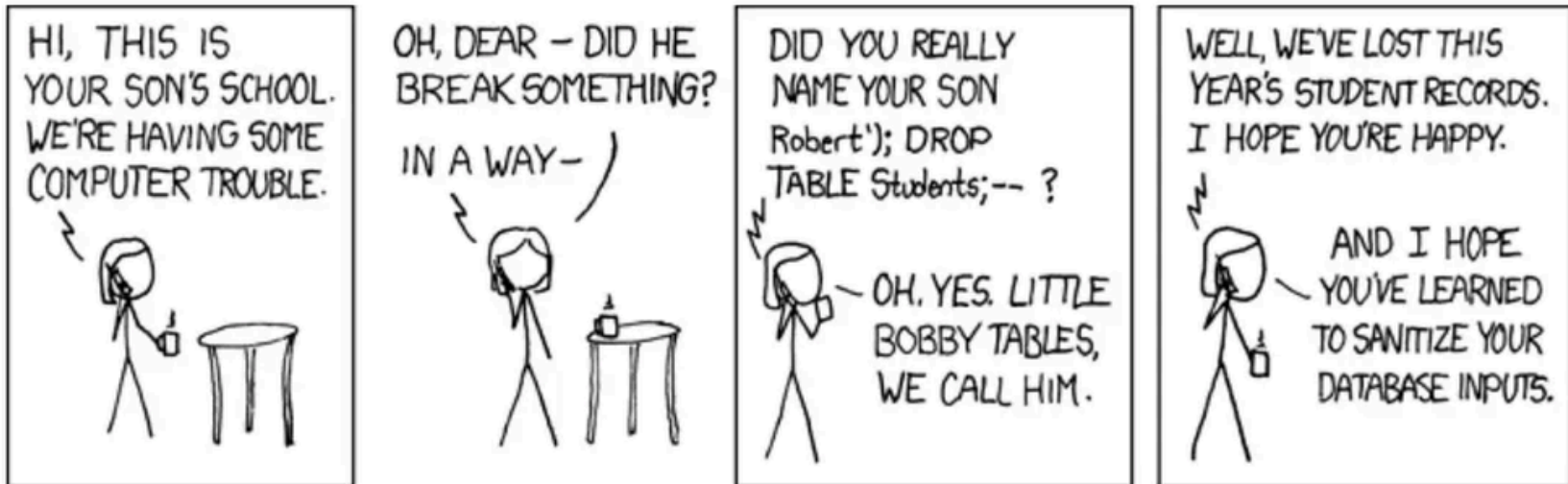
```
Statement st = con.createStatement();
st.executeUpdate("update STUDENT set NAME =" + student.getName()+ ...);

//pre-compiled (once) and DB-side caching
PreparedStatement pst =
    con.prepareStatement("update STUDENT set NAME = ? where ID = ?");
pst.setString(1, student.getName());
pst.setInt(2, student.getId());
pst.executeUpdate();
```

JDBC

Java DataBase Connectivity

```
Statement st = con.createStatement();  
st.executeUpdate("update STUDENT set NAME =" + student.getName()+ ...);
```



Gateway

Objecto que embrulha o acesso a uma determinada API de um recurso externo.



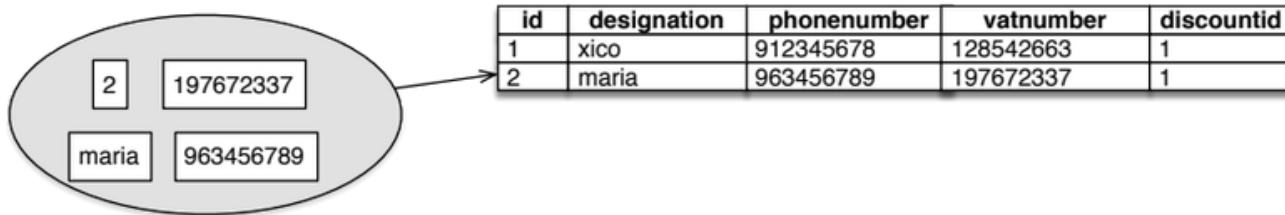
Gateways

Row Data Gateway

Table Data Gateway



Row Data Gateway



- Uma instância de uma classe corresponde a um registo.
- O objecto só tem lógica de edição de dados, não tem lógica de negócio!
- Funciona especialmente bem com o **Transaction Script**, na camada de negócio.

Row Data Gateway

Tabela => Classe
Registro => Instância

Customer
PK id : INTEGER
designation: VARCHAR(50)
phonenummer : INTEGER
vatnumber : INTEGER
discountid : INTEGER

CustomerRDGW
-id: int -vatNumber:int -designation:String -phoneNumber:int -discountId:int
+CustomerRDGW(int,String,int, DiscountType)
+insert() +delete() +update()
+getCustomerId():int +getDesignation():String +getDiscountType(): DiscountType +getPhoneNumber():int +getVatNumber():int +setDesignation(String) +setDiscountType(DiscountType) +setPhoneNumber(int)

Colunas => Atributos

Insert/Delete/Update
methods

Colunas => Getters + Setters

Row Data Gateway

For JDBC

```
public void insert () throws PersistenceException {  
    try (PreparedStatement statement = DataSource.INSTANCE.prepareGetGenKey(INSERT_CUSTOMER_SQL)) {  
        // set statement arguments  
        statement.setInt( parameterIndex: 1, vat);  
        statement.setString( parameterIndex: 2, designation);  
        statement.setInt( parameterIndex: 3, phoneNumber);  
        statement.setInt( parameterIndex: 4, discountId);  
        // executes SQL  
        statement.executeUpdate();  
        // Gets customer Id generated automatically by the database engine  
        try (ResultSet rs = statement.getGeneratedKeys()) {  
            rs.next();  
            id = rs.getInt( columnIndex: 1);  
        }  
    } catch (SQLException e) {  
        throw new PersistenceException ("Internal error!", e);  
    }  
}
```

Row Data Gateway

For JDBC

```
/**
 * insert a customer
 */
private static final String INSERT_CUSTOMER_SQL =
    "insert into " + TABLE_NAME +
    " (" + ID_COLUMN_NAME + ", " +
    VAT_NUMBER_COLUMN_NAME + ", " +
    DESIGNATION_COLUMN_NAME + ", " +
    PHONE_NUMBER_COLUMN_NAME + ", " +
    DISCOUNT_ID_COLUMN_NAME + ") " +
    "values (DEFAULT, ?, ?, ?, ?)";
```

Row Data Gateway

For JDBC

```
/**  
 * Columns' labels  
 */  
private static final String ID_COLUMN_NAME = "ID";  
private static final String VAT_NUMBER_COLUMN_NAME = "VATNUMBER";  
private static final String DESIGNATION_COLUMN_NAME = "DESIGNATION";  
private static final String PHONE_NUMBER_COLUMN_NAME = "PHONENUMBER";  
private static final String DISCOUNT_ID_COLUMN_NAME = "DISCOUNT_ID";
```

Row Data Gateway

For JDBC

```
public void insert () throws PersistenceException {
    try (PreparedStatement statement = DataSource.INSTANCE.prepareStatement(INSTALL_CUSTOMER_SQL)) {
        // set statement arguments
        statement.setInt( parameterIndex: 1, vat);
        statement.setString( parameterIndex: 2, designation);
        statement.setInt( parameterIndex: 3, phoneNumber);
        statement.setInt( parameterIndex: 4, discountId);
        // executes SQL
        statement.executeUpdate();
        // Gets customer Id generated automatically by the database engine
        try (ResultSet rs = statement.getGeneratedKeys()) {
            rs.next();
            id = rs.getInt( columnIndex: 1);
        }
    } catch (SQLException e) {
        throw new PersistenceException ("Internal error!", e);
    }
}

/**
 * insert a customer
 */
private static final String INSTALL_CUSTOMER_SQL =
    "insert into " + TABLE_NAME +
    " (" + ID_COLUMN_NAME + ", " +
    VAT_NUMBER_COLUMN_NAME + ", " +
    DESIGNATION_COLUMN_NAME + ", " +
    PHONE_NUMBER_COLUMN_NAME + ", " +
    DISCOUNT_ID_COLUMN_NAME + ") " +
    "values (DEFAULT, ?, ?, ?, ?)";

/**
 * Columns' labels
 */
private static final String ID_COLUMN_NAME = "ID";
private static final String VAT_NUMBER_COLUMN_NAME = "VATNUMBER";
private static final String DESIGNATION_COLUMN_NAME = "DESIGNATION";
private static final String PHONE_NUMBER_COLUMN_NAME = "PHONENUMBER";
private static final String DISCOUNT_ID_COLUMN_NAME = "DISCOUNT_ID";
```

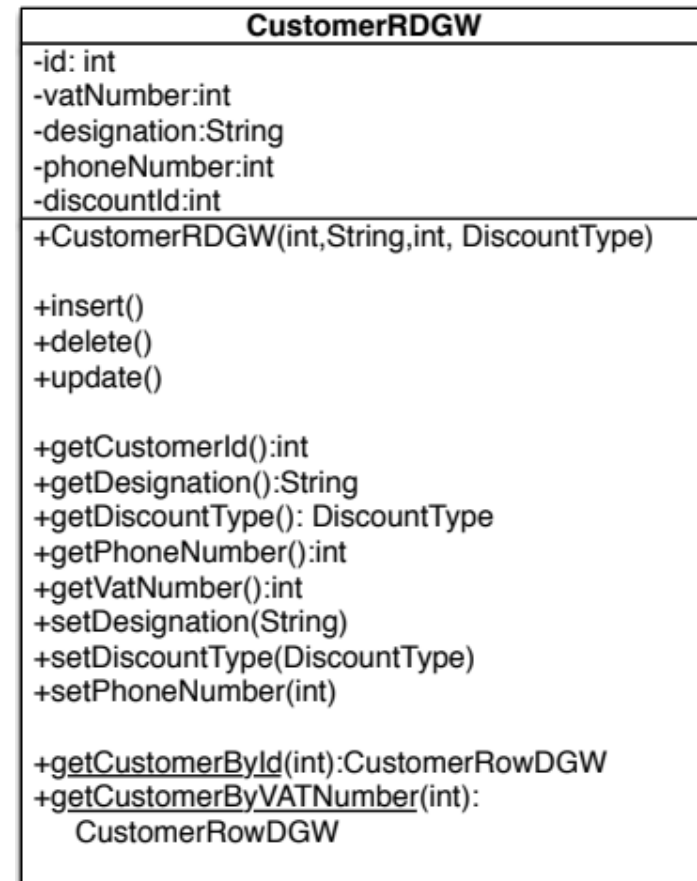
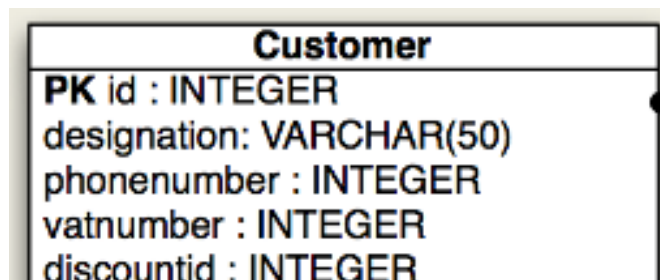
Row Data Gateway

Opções para procura em tabelas

- Métodos de classe (*static*) em *NomeDaTabelaRDGateway*.
 - Impede polimorfismo para reutilizar código em diferentes tipos de pesquisa (exemplo: paginação, ordenações por data, etc...)
- Métodos de instância de uma classe *NomeDaTabelaFinder*.
 - Classe focada na pesquisa (separada da representação dos dados)
 - A *NomeDaTabelaRDGateway* passa a poder ser gerada automaticamente

Row Data Gateway

Opção 1 (métodos *static*)



aka getCustomerById

```
public static CustomerRowDataGateway find (int id) throws PersistenceException {  
    try (PreparedStatement statement = DataSource.INSTANCE.prepare(GET_CUSTOMER_BY_ID_SQL)) {  
        // set statement arguments  
        statement.setInt(parameterIndex: 1, id);  
        // executes SQL  
        try (ResultSet rs = statement.executeQuery()) {  
            // creates a new customer from the data retrieved from the database  
            return loadCustomer(rs);  
        }  
    } catch (SQLException e) {  
        throw new PersistenceException("Internal error getting a customer by its id", e);  
    }  
}
```



```

public static CustomerRowDataGateway find (int id) throws PersistenceException {
    try (PreparedStatement statement = DataSource.INSTANCE.prepare(GET_CUSTOMER_BY_ID_SQL)) {
        // set statement arguments
        statement.setInt( parameterIndex: 1, id);
        // executes SQL
        try (ResultSet rs = statement.executeQuery()) {
            // creates a new customer from the data retrieved from the database
            return loadCustomer(rs);
        }
    } catch (SQLException e) {
        throw new PersistenceException("Internal error getting a customer by its id", e);
    }
}

```

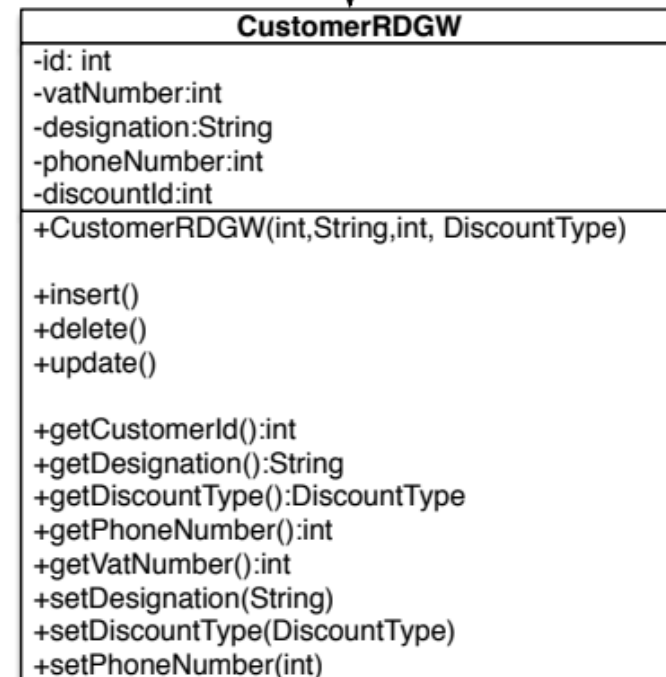
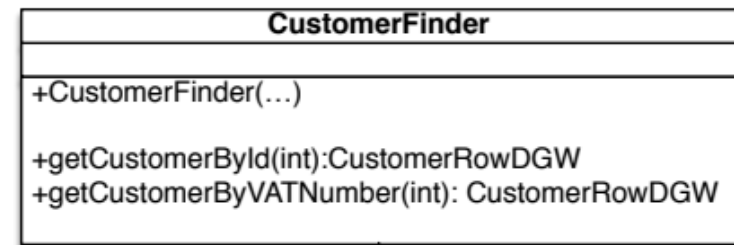
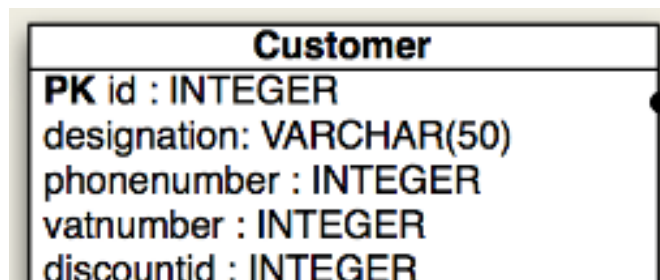
```

private static CustomerRowDataGateway loadCustomer(ResultSet rs) throws RecordNotFoundException {
    try {
        rs.next();
        CustomerRowDataGateway newCustomer = new CustomerRowDataGateway(rs.getInt( columnLabel: "vatnumber"),
            rs.getString( columnLabel: "designation"), rs.getInt( columnLabel: "phonenummer"),
            toDiscountType(rs.getInt( columnLabel: "discount_id")));
        newCustomer.id = rs.getInt( columnLabel: "id");
        return newCustomer;
    } catch (SQLException e) {
        throw new RecordNotFoundException ("Customer does not exist", e);
    }
}

```

Row Data Gateway

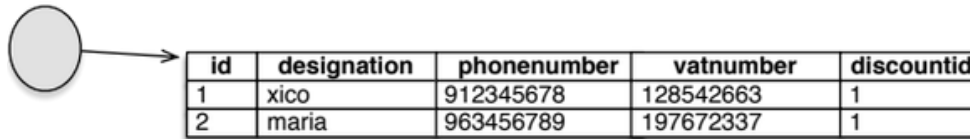
Opção 2 (classe dedicada)



Padrões para as várias camadas

Camada de Apresentação	MVC Front Controller Page Controller Page template
Camada de Negócio	Service Layer
	Domain Model
	Table Module
	Transaction Script
Camada de Dados	Row data gateway
	Table data gateway
	Active Record
	Data Mapper

Table Data Gateway



- Uma **instância** de uma classe corresponde a uma **tabela**.
- O objecto só tem lógica de edição de dados, não tem lógica de negócio!
- Funciona especialmente bem com o **Table Module**, mas também com **Transaction Script**, na camada de negócio.

Table Data Gateway

Tabela → Classe
Singleton!

Customer
PK id : INTEGER
designation: VARCHAR(50)
phonenumber : INTEGER
vatnumber : INTEGER
discountid : INTEGER

Insert, delete recebem parâmetros

CustomerTDGW
+find(int): ResultSet
+findByVATNumber(int): ResultSet ←
+insert(int,String,int,DiscountType)
+delete(int)
+updatePhoneNumber(int,int)
+updateDesignation(int,String)
+readId(resultSet):int
+readVatNumber(resultSet):int
+readDesignation(resultSet):String
+readPhoneNumber(resultSet):int
+readDiscountType(resultSet):DiscountType

ResultSet representa linhas

ResultSet é mais eficiente do que criar um objecto por linha; permite streaming!

```
public void insert (int vat, String designation, int phoneNumber, DiscountType discountType)
    throws PersistenceException {
    try (PreparedStatement statement = dataSource.prepare(INSERT_CUSTOMER_SQL)) {
        // set statement arguments
        statement.setInt( parameterIndex: 1, vat);
        statement.setString( parameterIndex: 2, designation);
        statement.setInt( parameterIndex: 3, phoneNumber);
        statement.setInt( parameterIndex: 4, discountType.ordinal());
        // execute SQL
        statement.executeUpdate();
    } catch (SQLException e) {
        throw new PersistenceException("Internal error adding the customer", e);
    }
}
```

```

public void insert (int vat, String designation, int phoneNumber, DiscountType discountType)
    throws PersistenceException {
    try (PreparedStatement statement = dataSource.prepare(INSERT_CUSTOMER_SQL)) {
        // set statement arguments
        statement.setInt(parameterIndex:1, vat);
        statement.setString( parameterIndex:2, designation);
        statement.setInt( parameterIndex:3, phoneNumber);
        statement.setInt( parameterIndex:4, discountType.ordinal());
        // execute SQL
        statement.executeUpdate();
    } catch (SQLException e) {
        throw new PersistenceException("Internal error adding the customer", e);
    }
}

```

```

public ResultSet find (int id) throws PersistenceException {
    try (PreparedStatement statement = dataSource.prepare(GET_CUSTOMER_BY_ID_SQL)) {
        // set statement arguments
        statement.setInt( parameterIndex:1, id);
        // executes SQL
        try (ResultSet rs = statement.executeQuery()) {
            return rs;
        }
    } catch (SQLException e) {
        throw new PersistenceException("Internal error obtaining the customer by id", e);
    }
}

```

```

public ResultSet find (int id) throws PersistenceException {
    try (PreparedStatement statement = dataSource.prepare(GET_CUSTOMER_BY_ID_SQL)) {
        // set statement arguments
        statement.setInt(parameterIndex: 1, id);
        // executes SQL
        try (ResultSet rs = statement.executeQuery()) {
            return rs;
        }
    } catch (SQLException e) {
        throw new PersistenceException("Internal error obtaining the customer by id", e);
    }
}

```

```

public static CustomerRowDataGateway find (int id) throws PersistenceException {
    try (PreparedStatement statement = DataSource.INSTANCE.prepare(GET_CUSTOMER_BY_ID_SQL)) {
        // set statement arguments
        statement.setInt(parameterIndex: 1, id);
        // executes SQL
        try (ResultSet rs = statement.executeQuery()) {
            // creates a new customer from the data retrieved from the database
            return loadCustomer(rs);
        }
    } catch (SQLException e) {
        throw new PersistenceException("Internal error getting a customer by its id", e);
    }
}

```


Table Data Gateway

+ eficiente (menos instâncias)

VS

Row Data Gateway

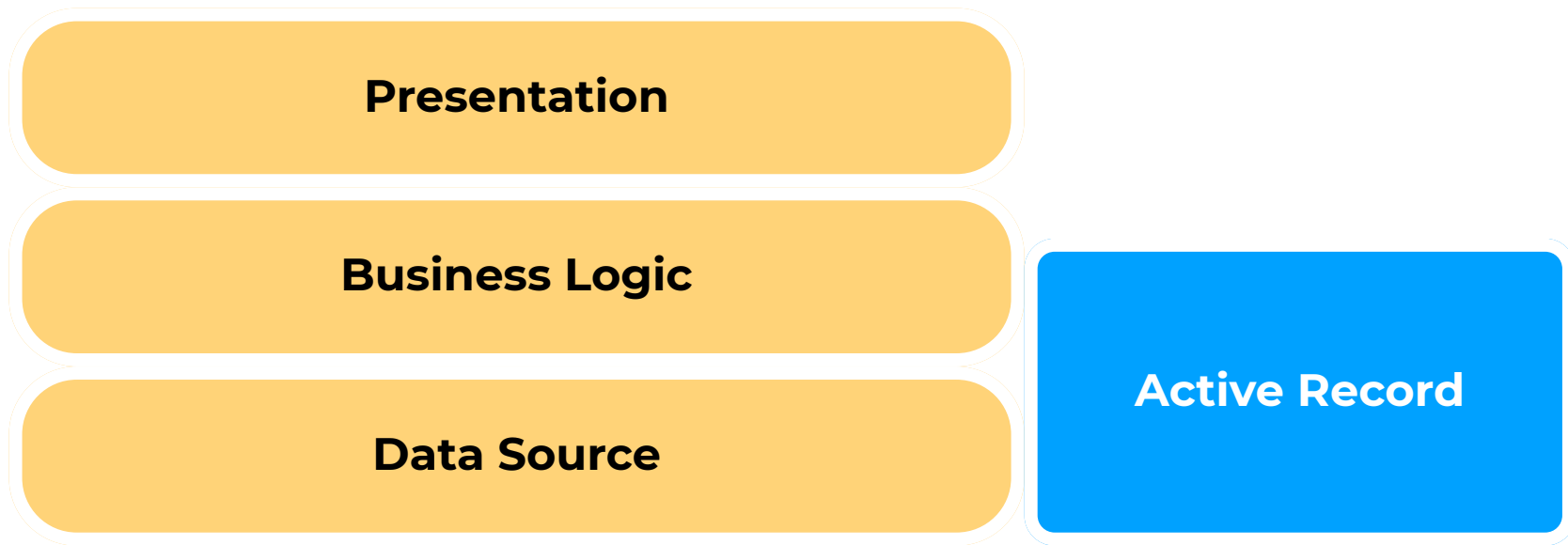
+ orientado a objectos

Padrões para as várias camadas

Camada de Apresentação	MVC Front Controller Page Controller Page template
Camada de Negócio	Service Layer
	Domain Model (+... ?)
	Table Module (+ TableDataGateway)
	Transaction Script (+RowDataGateway OR TableDataGateway)
Camada de Dados	Row data gateway
	Table data gateway
	Active Record
	Data Mapper

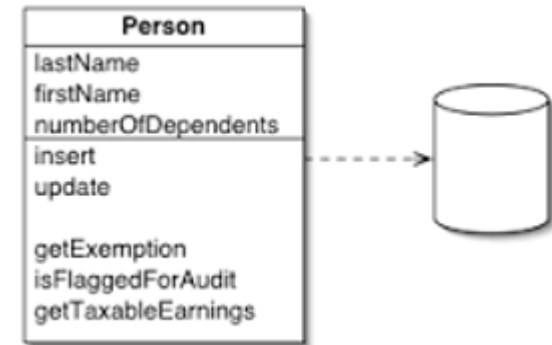
Padrões para as várias camadas

Camada de Apresentação	MVC Front Controller Page Controller Page template
Camada de Negócio	Service Layer
	Domain Model
	Table Module
	Transaction Script
Camada de Dados	Row data gateway
	Table data gateway
	Active Record
	Data Mapper



- ActiveRecord mistura lógica de negócio nas mesmas classes de acesso aos dados.
- Cada instância representa um registro de uma tabela, com métodos extra que adicionam lógica de negócio.
- Popularizada por Ruby on Rails.

Active Record



- Cada classe tem:
 - Um atributo por coluna da tabela
 - Construtores e getters/setters
 - Métodos de instância para:
 - `insert()`, `delete()`, `update()` para actualizar a base de dados
 - lógica de negócio associada com essa instância
 - Métodos de classe para:
 - Procurar instâncias
 - Carregar um `ResultSet` (evitando objectos duplicados)

Active Record

- Boa escolha quando o modelo de domínio e a base de dados estão bastante alinhados. (Domain model + RDGW)
 - Recomendado para CRUDs
 - Quando a lógica é complexa e envolve muitas classes, deixa de ser prático.
- **Desvantagens:**
 - As classes de negócio estão coupled com a API da DB, não seguindo a arquitectura em camadas.
 - Há dispersão de SQL e acesso a dados por várias classes

Padrões para as várias camadas

Camada de Apresentação	MVC Front Controller Page Controller Page template
Camada de Negócio	Service Layer
	Domain Model
	Table Module
	Transaction Script
Camada de Dados	Row data gateway
	Table data gateway
	Active Record
	Data Mapper

```
graph TD; P(Presentation) --- BL(Business Logic); BL --- DS(Data Source); DS --- DM(Data Mapper);
```

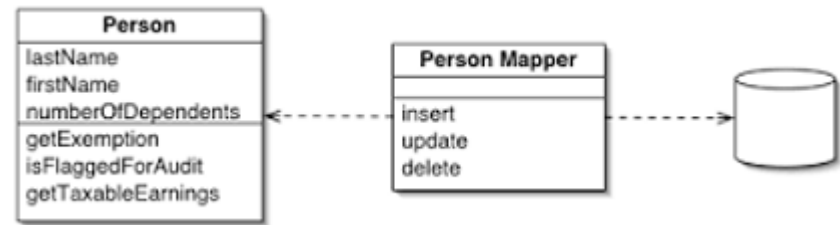
Presentation

Business Logic

Data Source

Data Mapper

Data Mapper



- Mappers são responsáveis por converter dados do modelo OOP para relacional e vice-versa.
- Um *mapper* transforma um grafo de objectos para registos em tabelas, e vice-versa, servindo de tradutor.
- Solução apropriada quando se usa Domain Model.
- Difícil de implementar bem, normalmente baseado em frameworks.